ZIMSEC

A LEVEL COMPUTER SCIENCE

REVISION NOTES + PRACTICE

Forms 5–6 | Heritage-Based Curriculum | 2024–2030

Short explanations • worked examples • labelled diagrams
Theory examination practice • practical preparation • answer guide

Candidate: _____

School: _____

Prepared as a concise learning aid. Always use the official syllabus and teacher guidance as the final authority.

How to use this revision booklet

These notes follow the ZIMSEC Computer Science Forms 5–6 syllabus for 2024–2030. The syllabus scope-and-sequence lists ten learning areas. Use the booklet to recall definitions, compare concepts, practise calculations and prepare practical skills.

Revision routine Read one topic, cover the page and explain the key terms from memory. Then attempt the practice questions without looking at the answer guide.

Syllabus coverage

No.	Learning area	High-value revision focus
1	Data representation	Number bases, two's complement, floating point, character codes, overflow and underflow
2	Computer architecture	Von Neumann/Harvard, buses, logic gates, registers, instruction cycle, pipelining, interrupts
3	Networking	OSI/TCP-IP, IP addressing, DNS, RIP/OSPF, cloud services and deployment
4	System Development Life Cycle	Waterfall, object-oriented methodology, prototyping
5	Security and ethics	Privacy, integrity, threats, access control, recovery, legislation, professional ethics
6	Algorithm design and data structures	Pseudocode, tracing, sorting/searching, arrays, trees, recursion
7	Programming	Programming paradigms, structured code, subprograms, arrays, files, OOP, interfaces
8	Databases	File-based systems, DBMS, relational modelling, keys, SQL, database management
9	Web application development	Client-side, server-side, database connectivity, CRUD, publishing
10	Enterprising	E-business, e-commerce, IPR, applications, telecommunications, business proposals

ZIMSEC assessment structure

Component	Format	Weighting
School-based continuous assessment	Two school-based projects (Forms 5 and 6); each contributes 10%	20%
Paper 1	Theory / free response; 3 hours; 100 marks; 10–12 compulsory questions	30%
Paper 2	Practical; 3 hours; 100 marks; one task from each of three sections	50%

Practical paper structure Web Application Development: 20 marks | Programming and Algorithm Design: 50 marks | Databases: 30 marks.

1. Data Representation

How computers encode numbers, text and other data using binary patterns.

Number systems

Base	Digits used	Place values
Binary (2)	0, 1	1, 2, 4, 8, 16, 32, 64, …
Octal (8)	0–7	1, 8, 64, 512, …
Denary (10)	0–9	1, 10, 100, 1000, …
Hexadecimal (16)	0–9, A–F	1, 16, 256, 4096, …

Hex digits: A=10, B=11, C=12, D=13, E=14, F=15.

Denary to binary: Repeatedly divide by 2 and read remainders from bottom to top, or use place values.

Binary to denary: Multiply each bit by its place value and add the results.

Binary to hexadecimal: Split bits into groups of four from the right; convert each group to one hex digit.

Worked example $(2F)_{16} = (0010\ 1111)_2 = 32 + 8 + 4 + 2 + 1 = 47_{10}$.

Signed integers: two's complement

- Positive values are written as ordinary binary with a leading 0 in a fixed-width representation.
- To form a negative value: write the positive binary value, invert every bit, then add 1.
- For n bits, the two's-complement range is $-2^{(n-1)}$ to $+2^{(n-1)}-1$. For 8 bits: −128 to +127.

Worked example: represent −23 in 8 bits +23 = 00010111. Invert = 11101000. Add 1 = 11101001. Therefore −23 = 11101001 in 8-bit two's complement.

Floating-point representation

A floating-point number stores a value using a sign, a mantissa (significand) and an exponent. The exponent positions the binary point; the mantissa stores the significant digits.

```
value = mantissa × 2^exponent
Example (normalised 0.1xxxx convention): 1101.01₂ = 0.110101₂ × 2^4
```

- Normalisation gives the mantissa a standard form and helps use the available bits efficiently.
- The exact sign-bit and mantissa convention must be taken from the representation specified in the question.
- A limited mantissa can cause rounding or truncation errors; not every decimal fraction has an exact finite binary representation.

Arithmetic errors and character codes

Term / code	Meaning
Overflow	A result is too large to fit in the available number of bits or exponent range.
Underflow	A non-zero result is too small in magnitude to be represented in the available range.
Rounding error	The value is approximated to the nearest representable value.
Truncation error	Extra digits are discarded rather than rounded.
ASCII	A character encoding with a relatively small basic character set.
Unicode	A character standard designed to represent characters from many writing systems.
BCD	Each decimal digit is encoded separately using four bits; e.g. 59 → 0101 1001.
EBCDIC	An 8-bit character encoding historically used on IBM mainframes.

Exam reminder Do not confuse overflow with underflow. Always show conversion workings and preserve the number of bits specified.

Quick recall checks

- Write 7 in 4-bit binary: 0111. In packed BCD, 47 is represented digit by digit as 0100 0111.
- For a fixed-width integer, check whether the result fits the permitted range before deciding whether overflow has occurred.
- When comparing character encodings, mention the range of characters supported and the number of bits / code points used where relevant.

2. Computer Architecture

How hardware components are organised and how the processor executes instructions.

Electronic components

Type	Description	Examples
Passive	Cannot provide power gain; they store, transfer or dissipate energy.	Resistor, capacitor, inductor
Active	Can control current or provide amplification / switching.	Diode, transistor, integrated circuit

Von Neumann and Harvard architectures

The Von Neumann model stores instructions and data in the same main memory and usually transfers both over a shared set of buses. Harvard architecture uses separate instruction and data memories / pathways.

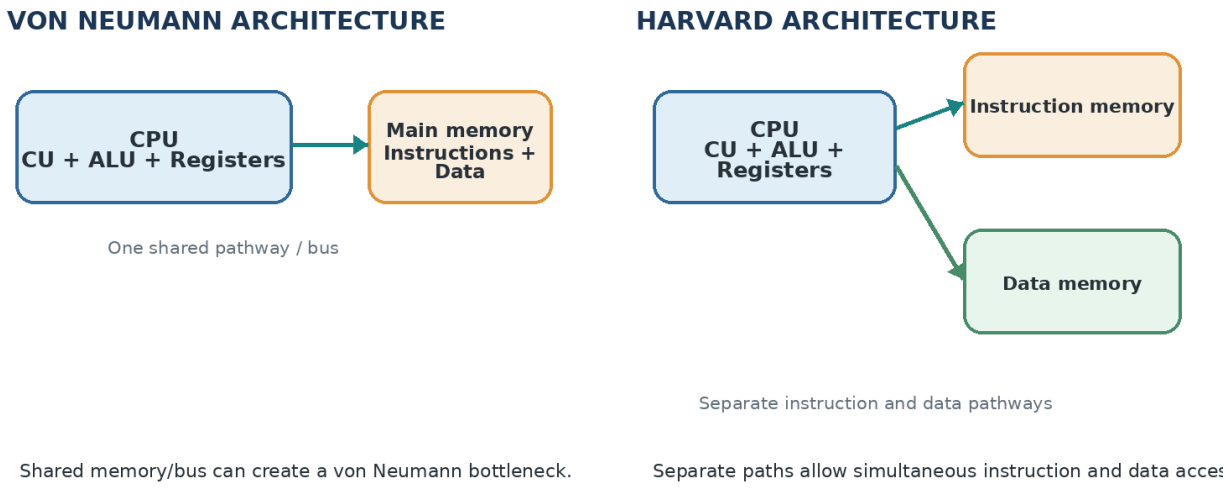


Figure 1. Shared instruction/data memory in Von Neumann; separate instruction and data paths in Harvard.

System buses

Bus	Carries	Direction / purpose
Address bus	Address of a memory or I/O location	Usually CPU → memory/device; width affects addressable locations.
Data bus	Actual data and instructions being transferred	Often bidirectional.
Control bus	Control and timing signals such as read, write and interrupt	Coordinates operations.

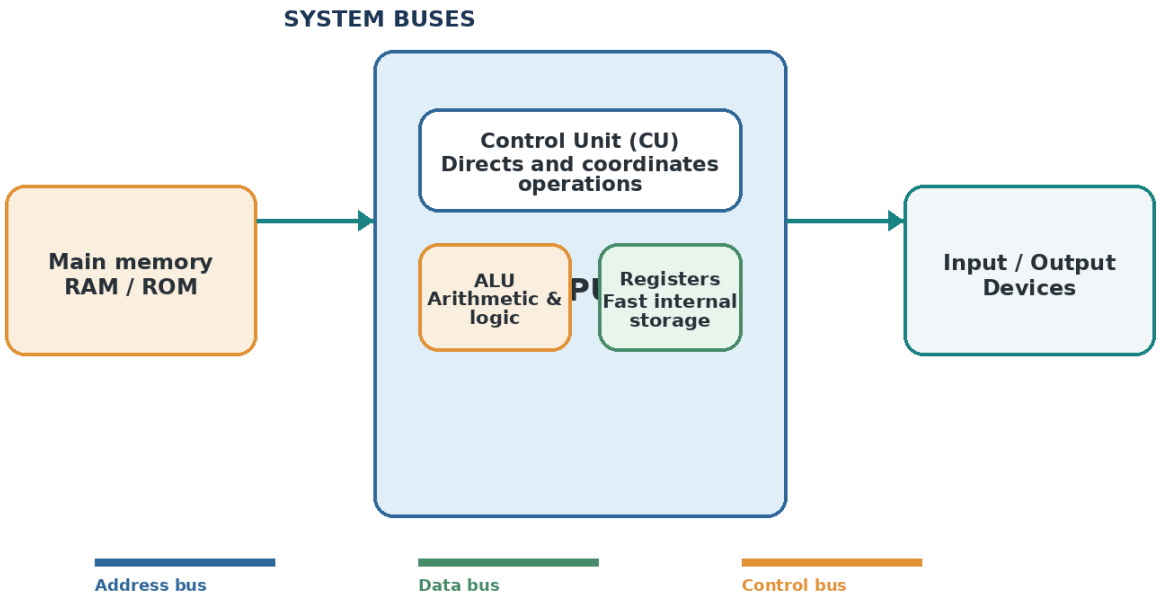


Figure 2. Simplified CPU, memory, I/O and system bus arrangement.

Processor components and registers

Component / register	Main function
Control Unit (CU)	Decodes instructions and coordinates processor activity.
Arithmetic Logic Unit (ALU)	Performs arithmetic and logical operations.
Program Counter (PC)	Holds the address of the next instruction.
Memory Address Register (MAR)	Holds the address of the memory location being accessed.
Memory Data Register (MDR)	Holds data being transferred to or from memory.
Current Instruction Register (CIR)	Holds the instruction currently being decoded / executed.
Accumulator	Stores intermediate arithmetic / logic results.
Index register	Supports indexed addressing and array / sequence access.
Status register	Stores flags such as zero, carry, negative or overflow.

Fetch–decode–execute cycle

1. Fetch: the address in the PC is copied to MAR; the instruction is read from memory into MDR, then copied to CIR.
2. Increment PC so it points to the next instruction (unless a branch changes it).
3. Decode: the CU interprets the opcode and determines the operands / addressing mode.
4. Execute: the required operation is performed; data may be read or written and flags updated.

Remember PC = where the next instruction is; MAR = which memory address is being accessed; MDR = data moving to or from memory; CIR = current instruction.

Pipelining, interrupts and addressing

- Pipelining overlaps stages of different instructions (for example, one instruction is decoded while another is fetched). It can improve throughput, although hazards and dependencies may cause stalls.
- An interrupt is a signal or event that causes the processor to pause its current sequence, save state, and execute an interrupt service routine. Types in the syllabus include internal, external and software interrupts.
- Addressing modes: immediate (value is in the instruction), direct (instruction gives the operand address), indirect (address points to another address), indexed (base address plus index).

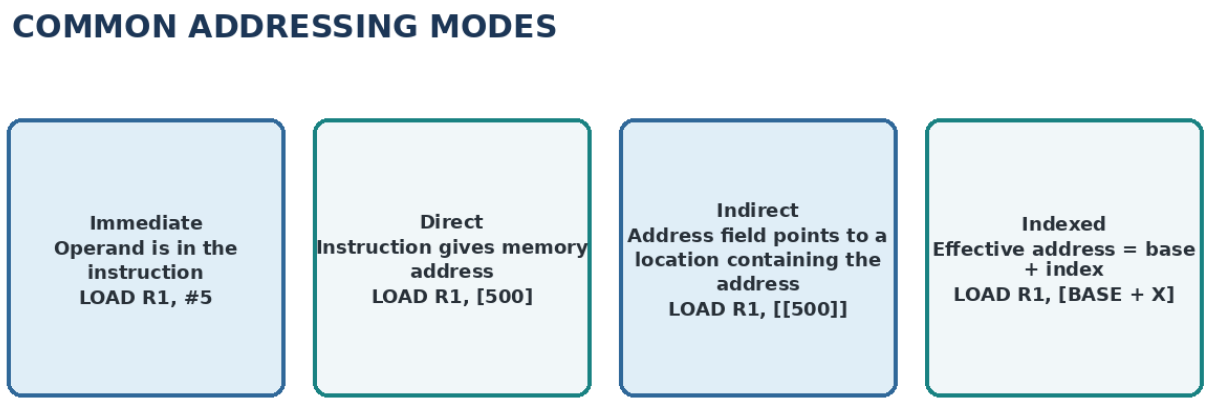


Figure 3. Addressing modes with simplified instruction examples.

Logic gates

A logic gate performs a Boolean operation on binary inputs. The syllabus includes NAND, NOR, XOR and XNOR in addition to basic AND, OR and NOT.

LOGIC GATE TRUTH TABLES (TWO INPUTS)

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

XOR = 1 when inputs differ. XNOR = 1 when inputs are the same.

Figure 4. Two-input truth tables. XOR is 1 when inputs differ; XNOR is 1 when they match.

Processor performance Clock speed, number of cores, cache size, word length, bus width, pipelining and processor architecture can affect performance. Clock speed alone does not determine overall performance.

3. Networking

How computers exchange data and how network services are delivered.

OSI reference model

Layer	Main role	Example / clue
7 Application	Services used by applications	HTTP, DNS, SMTP
6 Presentation	Translation, compression, encryption	Data format / encoding
5 Session	Establishes and manages communication sessions	Session control
4 Transport	End-to-end delivery, segmentation, reliability / ports	TCP, UDP
3 Network	Logical addressing and routing	IP, routers
2 Data Link	Frames, MAC addressing, link-level error detection	Ethernet, switches
1 Physical	Signals, cables, radio and bit transmission	Copper, fibre, Wi-Fi radio

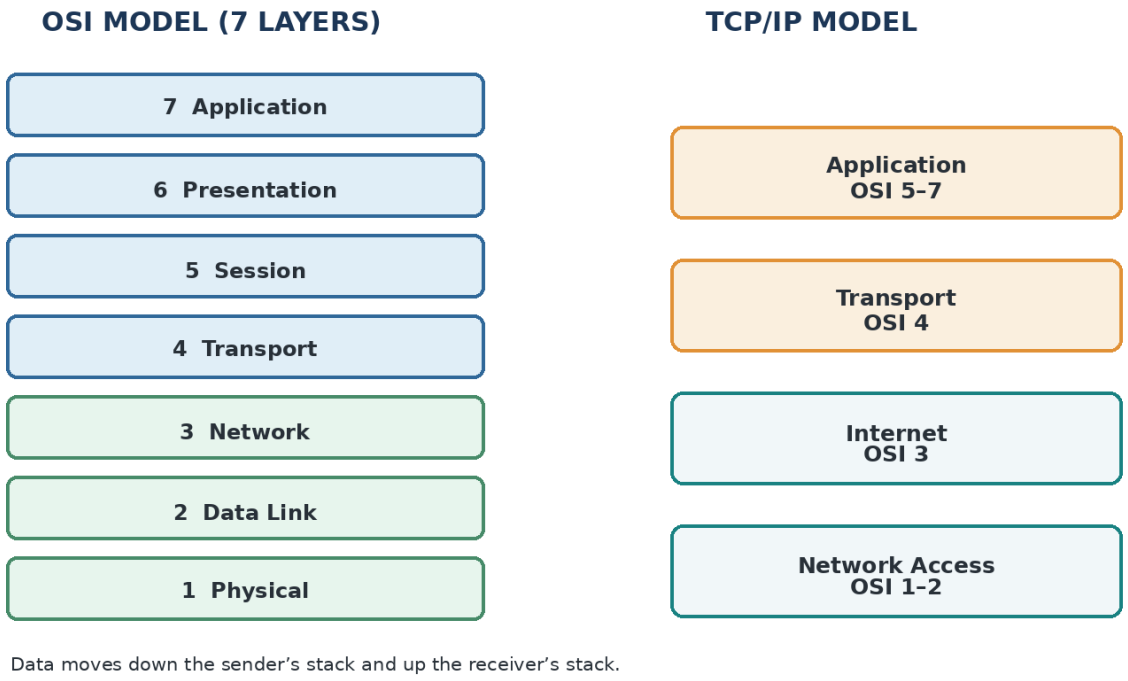


Figure 5. OSI layers and a common four-layer TCP/IP grouping.

TCP/IP, addressing and DNS

- TCP/IP is the protocol suite used for communication across interconnected networks. TCP provides reliable, ordered delivery; IP handles addressing and packet routing.
- An IPv4 address is 32 bits, commonly written as four decimal octets (for example, 192.168.1.10). A subnet mask / prefix identifies the network portion and host portion.
- Public IP addresses are globally routable; private IPv4 addresses are used within private networks and normally reach the internet through NAT.
- DNS translates human-readable domain names into IP addresses using DNS records and name servers.

Private IPv4 address ranges

Private IPv4 range	Address range
10.0.0.0/8	10.0.0.0 – 10.255.255.255
172.16.0.0/12	172.16.0.0 – 172.31.255.255
192.168.0.0/16	192.168.0.0 – 192.168.255.255

Routing protocols

Protocol	How it works	Key comparison
RIP	Distance-vector protocol; uses hop count as its metric; maximum usable route length is 15 hops.	Simple but slower to converge and less scalable.
OSPF	Link-state protocol; builds a topology view and calculates shortest paths using a cost metric.	Generally faster convergence and better scalability in larger networks.

Cloud computing

Service model	What the customer receives	Example idea
IaaS	Virtual machines, storage and networking; customer manages OS and applications.	Rent a virtual server.
PaaS	Managed platform / runtime for developing and deploying applications.	Deploy code without managing the underlying servers.
SaaS	Ready-to-use software accessed over a network.	Webmail or online office suite.

Deployment type	Meaning
Public cloud	Services shared across customers and run by a cloud provider.
Private cloud	Cloud infrastructure dedicated to one organisation.
Hybrid cloud	Combination of private and public cloud environments.
Community cloud	Shared by organisations with common requirements.

Exam comparison Do not mix up a cloud service model (IaaS/PaaS/SaaS) with a deployment type (public/private/hybrid/community).

4. System Development Life Cycle (SDLC)

A structured approach to investigating, designing, building and maintaining information systems.

Waterfall model

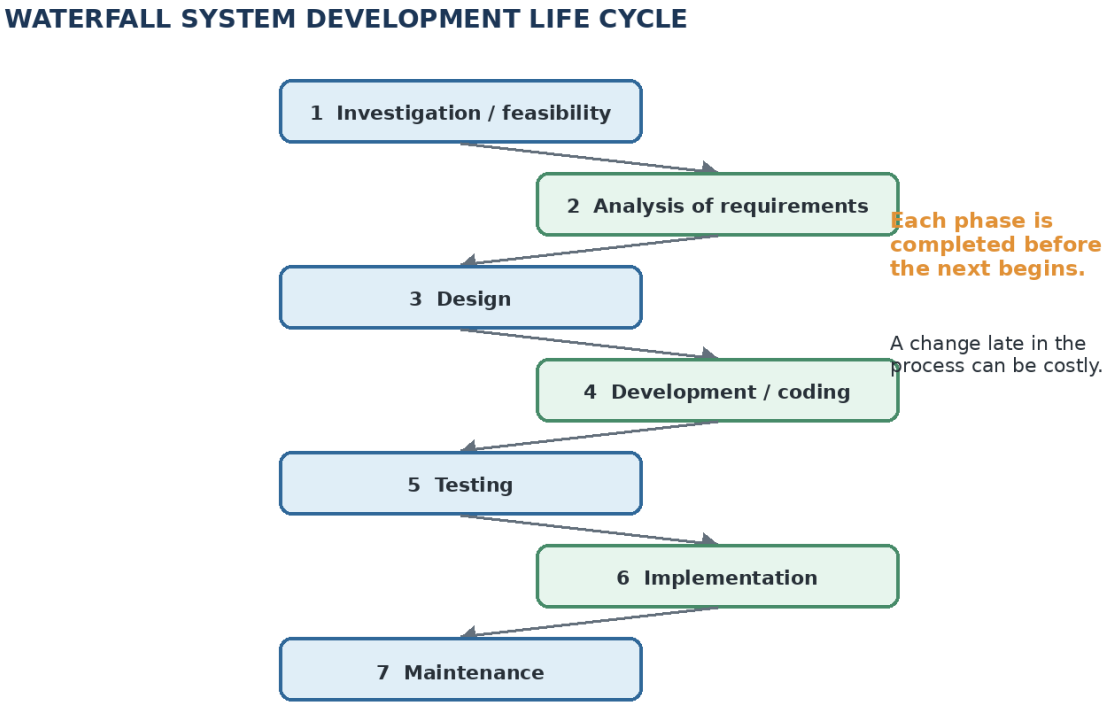


Figure 6. Typical waterfall stages. Some organisations use different names or combine stages.

Stage	Typical activities / outputs
Investigation / feasibility	Define problem, scope, objectives; assess technical, economic, legal, operational and schedule feasibility.
Analysis	Gather and document user requirements; examine current system and identify constraints.
Design	Specify data structures, inputs, outputs, interfaces, processes, architecture and security.
Development	Write code, configure hardware / software and create databases.
Testing	Test modules and the complete system; correct defects; check against requirements.
Implementation	Install, migrate data, train users and move to the new system.
Maintenance	Fix errors, adapt to changes and improve performance / features.

Object-oriented methodology (OOM)

- Model a system as interacting objects that combine data (attributes) and behaviour (methods).
- Common analysis / design artefacts include use-case diagrams, class diagrams and sequence diagrams; each communicates a different view of the system.

Prototyping and object-oriented design diagrams

In object-oriented analysis and design, diagrams make requirements and relationships easier to communicate. A class diagram describes the static structure of classes; a use-case diagram shows actors and the goals they perform; a sequence diagram shows messages exchanged in time order.

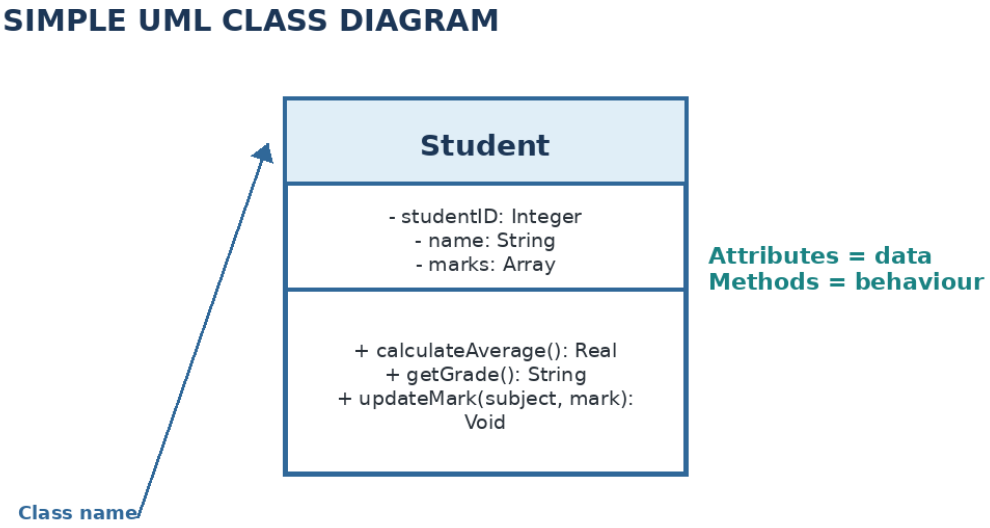


Figure 7. Example class diagram: attributes describe stored state and methods describe behaviour.

Diagram	What it communicates
Use-case	Actors outside the system and the services / goals they interact with.
Class	Classes, attributes, methods and relationships such as association or inheritance.
Sequence	Objects / participants and the time-ordered messages between them.

Prototyping

A prototype is an early working model used to explore requirements or test a design. Users give feedback, and the model is refined.

Advantages	Limitations
Users can see and comment on a tangible model.	Users may mistake the prototype for a finished system.
Unclear requirements can be discovered early.	Frequent changes can increase cost or cause scope creep.
Interface and usability can be evaluated early.	Quick prototype code may be unsuitable for the final system.

Compare carefully Waterfall is sequential and documentation-driven. Prototyping is iterative and feedback-driven. Choose a method according to requirement clarity, risk, user involvement and project constraints.

5. Security and Ethics

Protect information and systems while using technology legally and responsibly.

Confidentiality, integrity and availability

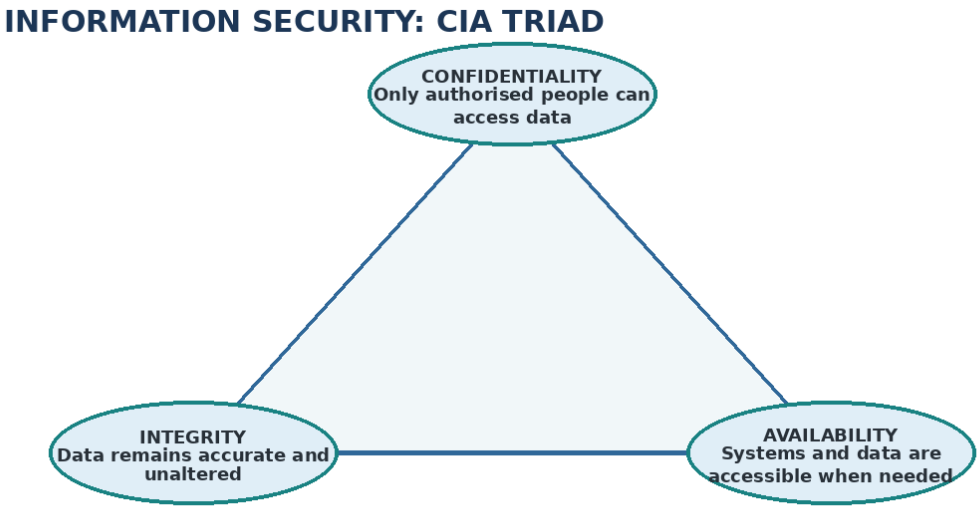


Figure 7. The CIA triad: three core information-security objectives.

Concept	Meaning	Example control
Confidentiality / privacy	Only authorised users should see personal or sensitive data.	Access permissions, encryption, strong authentication.
Integrity	Data should remain accurate, complete and protected from unauthorised change.	Hashes, checks, audit logs, input validation.
Availability	Systems and data should be accessible when required.	Backups, redundancy, patching, disaster recovery.

Threats and vulnerabilities

Threat	Description	Control / mitigation
Malware	Malicious software such as viruses, worms, ransomware or spyware.	Endpoint protection, patching, least privilege, backups.
Phishing / social engineering	Tricking users into disclosing information or taking unsafe actions.	Awareness training, MFA, verify links and requests.
Unauthorised access	An attacker or unauthorised insider accesses a system.	Strong authentication, access control, monitoring.
Denial of service	Overwhelming a service so legitimate users cannot access it.	Filtering, rate limits, resilient infrastructure.
Insider threat	A trusted user misuses access deliberately or accidentally.	Least privilege, separation of duties, logging.
Unpatched software	Known weaknesses remain exploitable.	Updates, vulnerability management, testing.

Protection, recovery and risk management

- Authentication verifies identity; authorisation determines what an authenticated user may do.
- Multi-factor authentication combines different factor types: something you know, have or are.
- Encryption protects confidentiality; backups support recovery; a firewall controls network traffic; anti-malware detects or blocks malicious code.
- A disaster recovery plan defines how systems and data will be restored after a serious disruption. Keep tested backups, including an offline or isolated copy where practical.
- Risk management: identify assets and threats, assess likelihood and impact, choose controls, assign responsibility, monitor and review.

Authentication factors and access control

Factor type	What it means	Example
Something you know	Secret information	Password, PIN
Something you have	A physical or digital item	Smart card, authenticator token
Something you are	A biometric characteristic	Fingerprint, face recognition

Role-based access control (RBAC) assigns permissions according to a user's job role. The principle of least privilege gives each user only the access needed to perform their duties.

Ethics, legislation and environmental impact

- Respect privacy, obtain informed consent, protect confidential information and use data only for legitimate purposes.
- Observe copyright and intellectual property rights. Do not plagiarise, pirate software or disclose confidential source code.
- Professional ethics include honesty, competence, accountability, respect, fairness and avoiding harm.
- Social media can support communication and marketing but can also spread misinformation, enable cyberbullying, expose private data and damage reputations.
- E-waste contains materials that can harm people and the environment if discarded carelessly. Apply the 3Rs: Reduce, Reuse and Recycle. Use authorised collection / recycling channels.

Zimbabwe context For legal questions, use the relevant Zimbabwean legislation and the terminology taught by your school. Be able to explain the purpose of data-protection, cybercrime and copyright rules; do not rely on outdated legal summaries.

Risk analysis mini-example

Asset	Threat / weakness	Possible control
Learner records	Phishing email steals a staff password.	MFA, awareness training, least-privilege access.
School server	Disk failure or ransomware.	Tested offline backup, patching, recovery procedure.
Online portal	Personal details exposed through weak permissions.	Access review, encryption, logging and secure testing.

6. Algorithm Design and Data Structures

Design clear, finite steps for solving problems and choose suitable ways to store data.

Pseudocode and control structures

Structure	Purpose	Typical keywords
Sequence	Instructions run in order.	INPUT, OUTPUT, assignment
Selection	Choose a path according to a condition.	IF…THEN…ELSE, CASE
Iteration	Repeat instructions.	FOR, WHILE, REPEAT…UNTIL

Good algorithm A well-designed algorithm has clear inputs, defined outputs, unambiguous steps, finite termination and an effective method.

Trace tables and flowcharts

- A trace table records variable values after each important statement or loop iteration. Update values in execution order.
- A flowchart uses standard symbols: oval = start/end; parallelogram = input/output; rectangle = process; diamond = decision; arrows = flow direction.

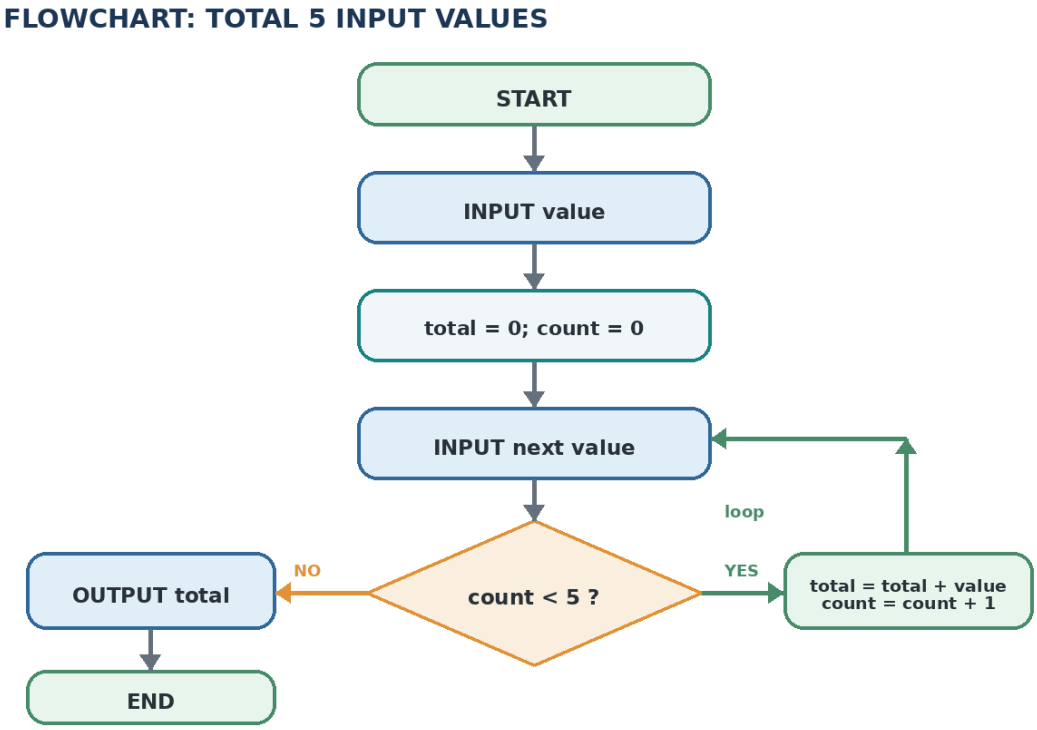


Figure 8. Example flowchart. The NO branch exits after five values; the YES branch updates and repeats.

Searching algorithms

Algorithm	Method	Important condition / performance
Linear search	Checks each item in turn until found or list ends.	Works on unsorted data; worst-case time grows with n.
Binary search	Checks the middle item and repeatedly halves the search interval.	Data must be sorted; worst-case time grows about $\log_2(n)$.

Sorting algorithms

- Bubble sort compares adjacent values and swaps out-of-order pairs. Passes continue until no swaps are needed or the list is sorted.

- Quick sort selects a pivot, partitions values around it, then recursively sorts the partitions. Average performance is O(n log n); worst case is $O(n^2)$ with poor pivots.

```
Bubble sort (ascending)
FOR pass = 1 TO n - 1
    swapped = FALSE
    FOR i = 0 TO n - pass - 1
        IF A[i] > A[i + 1] THEN
            SWAP A[i], A[i + 1]
            swapped = TRUE
        ENDIF
    NEXT i
    IF swapped = FALSE THEN EXIT FOR
NEXT pass
```

Data structures

Structure	Characteristics	Suitable use
Array (static)	Indexed collection; usually fixed size and elements of a defined type.	Fast direct access by index.
Binary tree (dynamic)	Nodes linked by references; each node has at most two children.	Hierarchical data and search structures.
Stack	Last In, First Out (LIFO).	Undo operations, function calls.
Queue	First In, First Out (FIFO).	Print jobs, waiting-line processing.

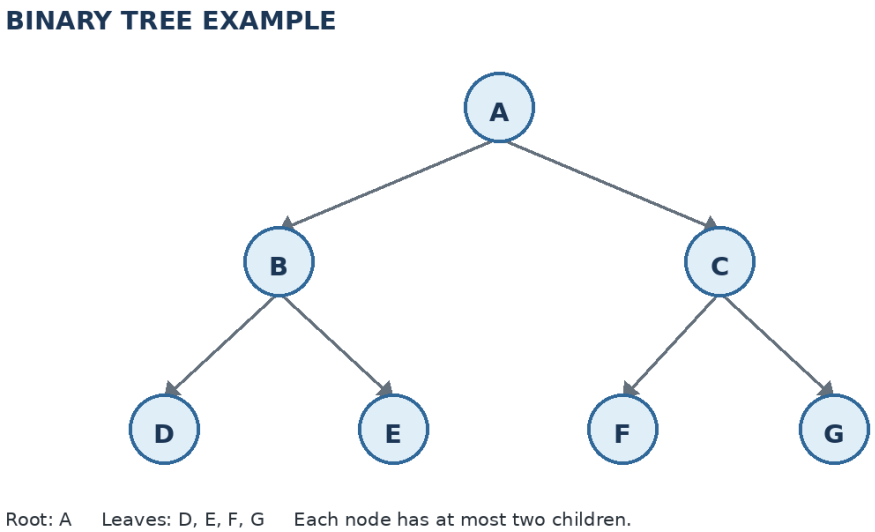


Figure 9. A binary tree. A is the root; D, E, F and G are leaf nodes.

Recursion and tracing recursive calls

Recursion occurs when a routine calls itself. It must have a base case that stops the calls and a recursive case that makes progress towards that base case.

```
FUNCTION factorial(n)
    IF n = 0 THEN
        RETURN 1          // base case
    ELSE
        RETURN n * factorial(n - 1)
    ENDIF
ENDFUNCTION
```

Common error A recursive routine without a reachable base case can continue calling itself until the call stack is exhausted.

Call	Action	Return value
factorial(3)	3 × factorial(2)	6
factorial(2)	2 × factorial(1)	2
factorial(1)	1 × factorial(0)	1
factorial(0)	Base case returns 1	1

- Each call has its own local parameters and waits for the smaller call to return.
- After the base case returns, calls unwind in reverse order and combine their results.
- Recursion can make divide-and-conquer algorithms clear, but it uses call-stack memory. An iterative solution may be more memory-efficient for some tasks.

7. Programming

Use programming language features and structured techniques to develop reliable solutions.

Programming paradigms and language features

Paradigm / feature	Summary
Imperative / procedural	Describes how to perform a task using statements that change program state.
Declarative	Describes what result is required rather than every step for producing it.
Object-oriented	Organises a program around objects created from classes.
General-purpose language	Can be used for many types of applications.
Special-purpose language	Designed for a narrower task or domain.

- A variable stores a value that may change; a constant represents a value that should not change during execution.
- An expression produces a value; a statement performs an action.
- Data types describe the kind of values a variable can hold, such as integer, real, Boolean, character and string.
- Structured programming uses sequence, selection and iteration, with meaningful modules and controlled flow.

Functions, procedures and parameter passing

Term	Meaning
Function	A named subprogram that returns a value.
Procedure	A named subprogram that performs an action and may not return a value.
Parameter	A variable in a subprogram definition that receives an input.
Argument	The actual value or variable passed to a subprogram.
Pass by value	The subprogram receives a copy; changing the parameter does not change the caller's variable.
Pass by reference	The subprogram receives access to the caller's variable; changes can affect the original.

Arrays and file handling

- An array stores multiple related values under one identifier, accessed using an index. Check array bounds and whether indexing starts at 0 or 1 in the chosen language.
- Sequential file access processes records in order. Random/direct access can move to a specified position when the file structure supports it.
- Typical file operations: open, read, write / append, close. Handle missing files and invalid data safely.

```
OPEN "marks.txt" FOR INPUT
WHILE NOT EOF(file)
    READ file, mark
    total = total + mark
ENDWHILE
CLOSE file
```

Object-oriented programming (OOP)

Concept	Meaning	Example
Class	Blueprint defining attributes and methods.	Student class
Object	An instance of a class.	student1
Encapsulation	Combines data and methods and controls access to internal state.	Private balance with deposit method
Inheritance	A class derives properties / methods from another class.	PostgraduateStudent extends Student
Polymorphism	The same interface / method name can behave differently for different object types.	draw() for Circle and Rectangle

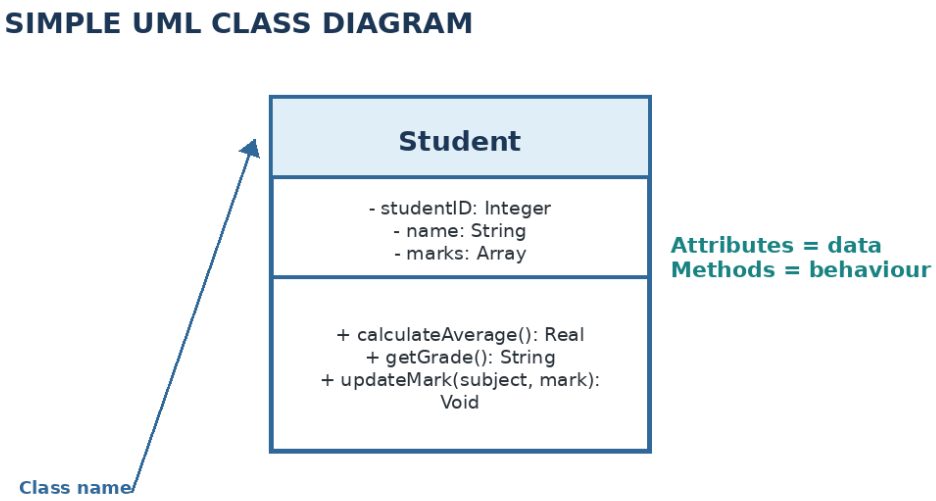


Figure 10. UML class notation: a class name, attributes and methods.

Design principle Keep data and behaviour together where appropriate, hide internal implementation details, and make classes cohesive. Avoid unnecessary inheritance.

Interface design

- Design for the intended user: readable text, consistent controls, clear navigation, useful feedback and accessible colour contrast.
- Prevent errors with validation, helpful error messages, confirmation for destructive actions and sensible defaults.
- Separate presentation, application logic and data access where possible.

8. Databases

Organise data so it can be stored, queried, updated and protected consistently.

File-based systems and DBMS

File-based approach	Database management system
Separate application files can duplicate the same data.	Centralised managed data can reduce duplication.
Program-data dependence can make changes difficult.	Data independence separates data structure from individual applications.
Limited concurrency and security controls.	Supports transactions, concurrent users, permissions and recovery.
Relationships and integrity are harder to enforce.	Constraints and relationships help maintain integrity.

Relational database concepts

Term	Meaning
Table / relation	A set of related records organised into rows and columns.
Record / tuple	One row describing an entity instance.
Field / attribute	A named property stored in a column.
Primary key (PK)	Field or combination of fields that uniquely identifies each row.
Foreign key (FK)	Field that references a key in another table and helps enforce relationships.
Candidate key	A minimal field set that could uniquely identify each row.

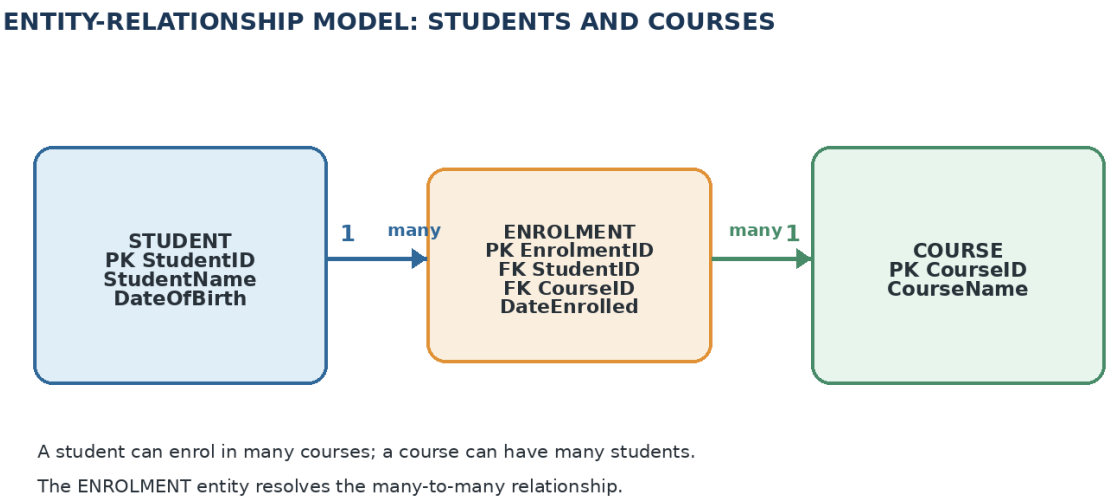


Figure 10. A many-to-many Student–Course relationship resolved through ENROLMENT.

Database normalisation

Form	Rule to remember
1NF	Each cell holds one atomic value; no repeating groups; rows are uniquely identifiable.
2NF	In 1NF, and every non-key attribute depends on the whole candidate key, not just part of a composite key.
3NF (background)	In 2NF, with no transitive dependency of non-key attributes on a key.

Example If an ENROLMENT table has composite key (StudentID, CourseID) but StudentName depends only on StudentID, move student details to STUDENT. If CourseName depends only on CourseID, move course details to COURSE.

SQL essentials

Command group	Examples	Purpose
DDL	CREATE, ALTER, DROP	Define / change database structures.

DML	INSERT, UPDATE, DELETE	Change records.
Query	SELECT, FROM, WHERE, ORDER BY, GROUP BY	Retrieve and summarise data.

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(60) NOT NULL,
    FormLevel INT
);

SELECT StudentName, FormLevel
FROM Student
WHERE FormLevel = 6
ORDER BY StudentName;
```

```
INSERT INTO Student (StudentID, StudentName, FormLevel)
VALUES (101, 'Tariro Moyo', 6);

UPDATE Student SET FormLevel = 6 WHERE StudentID = 101;
DELETE FROM Student WHERE StudentID = 101;
```

CRUD and database management

- CRUD = Create, Read, Update, Delete. These are the four basic data operations used by many applications.
- Use constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK) to protect data quality.
- Use transactions to group related changes; use backups, access permissions and audit trails to protect the database.

SQL safety When an application accepts user input, use parameterised queries / prepared statements rather than joining raw input into SQL strings.

9. Web Application Development

Create websites and applications that run through a browser and may connect to a server and database.

Client-side and server-side development

Technology	Typical role
HTML	Defines the structure and meaning of page content.
CSS	Controls layout, colours, typography and responsive appearance.
JavaScript	Adds client-side behaviour and interaction.
Server-side language	Processes requests, applies business rules, validates data and accesses the database.
Database	Stores persistent application data.

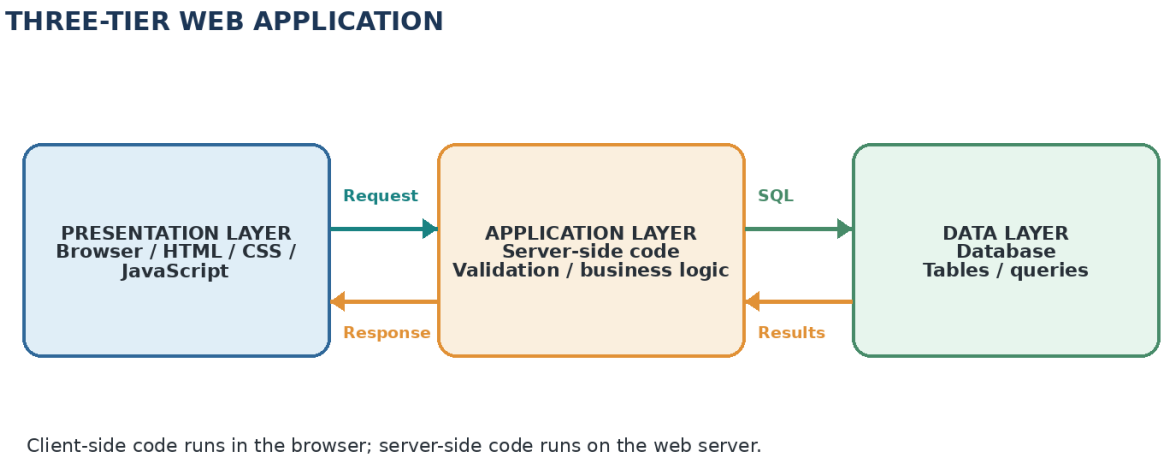


Figure 11. Typical three-tier web application. Requests and responses travel between layers.

Simple HTML page and form

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Registration</title>
</head>
<body>
  <h1>Student Registration</h1>
  <form action="register" method="post">
    <label for="name">Full name</label>
    <input id="name" name="name" required>
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

- GET requests normally retrieve data; POST requests normally submit data for processing. Sensitive information should not be placed in a URL.
- Client-side validation improves usability, but server-side validation is essential because browser checks can be bypassed.
- A website should be tested in different browsers and screen sizes. Publish using a web server / hosting service and configure the domain and security certificate where required.

Requests, validation, publishing and CRUD

1. Browser sends a request to the web server.
2. Server-side code validates the request and checks authorisation.
3. Application code sends a parameterised query to the database.
4. Database returns results; server constructs a response.
5. Browser displays the response and reports any errors clearly.

Validation checklist

Validation	Example
Presence check	A required name field must not be left blank.
Type check	Age must be numeric / date field must contain a date.
Range check	A mark must be within 0–100.
Format check	Email address must follow an expected pattern.
Length check	Phone number / ID must meet the required length.

Must know CRUD means Create, Retrieve/Read, Update and Delete. The practical syllabus expects candidates to create/connect a database, set up tables, perform CRUD operations and publish a website.

10. Enterprising

Apply computing knowledge to business opportunities and responsible innovation.

E-business and e-commerce

Term	Meaning	Example
E-business	Use of digital systems to support a wide range of business processes.	Online stock control, digital HR, supplier portals.
E-commerce	Buying and selling goods or services electronically.	Online shop accepting orders and payments.
Telecommunications	Electronic transmission of information over distance.	Mobile networks, fibre, satellite, VoIP.

Applications of Computer Science

- Engineering and manufacturing: CAD, simulation, robotics and process control.
- Agriculture: precision farming, irrigation monitoring, mechanisation and market information.
- Research and development: modelling, data analysis and high-performance computing.
- Education: e-learning, digital assessment and learning management systems.
- Finance and commerce: digital payments, accounting, fraud detection and customer services.
- Health, transport, mining, environment and public services: information systems, monitoring and decision support.

Intellectual property rights (IPR)

Right / concept	What it protects / means
Copyright	Original creative works such as software code, text, music, images and documentation.
Patent	A qualifying invention, subject to legal requirements and registration.
Trademark	A sign / name / logo distinguishing goods or services.
Trade secret	Confidential business information that provides commercial value.
Plagiarism	Presenting someone else's work or ideas as your own without proper acknowledgement.

Business proposal essentials

1. Executive summary: what the business will offer and why it is viable.
2. Problem / opportunity and target customers.
3. Product or service description and unique value.
4. Market research and competitor analysis.
5. Marketing and sales strategy (product, price, place, promotion).
6. Operations: people, equipment, suppliers, technology and delivery.
7. Financial plan: start-up costs, revenue forecast, expenses and break-even estimate.
8. Risks, legal / ethical responsibilities and implementation timeline.

Exam approach When asked to evaluate a business idea, discuss benefits, costs, customers, competition, infrastructure, security, legal duties and risks - then reach a reasoned conclusion.

Theory practice paper

ZIMSEC A Level Computer Science 6023/1 style • 3 hours • 100 marks • Answer all questions. These are original revision questions, not official ZIMSEC questions.

Instructions Show all calculations. Use correct technical terminology. Where a question asks you to explain, give a point and state how or why it matters.

Question 1 - Data representation [12 marks]

(a) Convert $(2F)_{16}$ into (i) denary and (ii) 8-bit binary. [4]

(b) Represent −23 using 8-bit two's complement. Show your working. [3]

(c) Explain the purpose of the mantissa and exponent in floating-point representation. [3]

(d) Distinguish between overflow and underflow. [2]

Question 2 - Computer architecture [12 marks]

(a) State two differences between Von Neumann and Harvard architectures. [4]

(b) Explain the role of the address bus, data bus and control bus. [3]

(c) Describe the purpose of the PC, MAR, MDR and CIR. [4]

(d) State one benefit of pipelining. [1]

Question 3 - Networking [12 marks]

(a) Name the seven OSI layers in the correct order, from application to physical. [3]

(b) Explain the role of DNS. [2]

(c) State one difference between public and private IPv4 addresses. [2]

(d) Compare RIP and OSPF using their routing approach or metric. [3]

(e) Distinguish SaaS from IaaS. [2]

Question 4 - SDLC [8 marks]

(a) Describe four stages of the Waterfall SDLC and one key output from each. [4]

(b) Give two benefits and two limitations of prototyping. [4]

Question 5 - Security and ethics [8 marks]

(a) Explain confidentiality, integrity and availability. [3]

(b) A school employee receives an email asking for their password through a link. Identify the threat and give two controls. [3]

(c) State two measures that should appear in a disaster recovery plan. [2]

Question 6 - Algorithms and data structures [12 marks]

(a) A sorted array contains [3, 7, 11, 18, 24, 31, 45, 52]. Use binary search to show the midpoint values checked when searching for 31. [4]

(b) Show the array after the first complete pass of bubble sort in ascending order: [9, 4, 7, 2]. [3]

(c) Give two differences between a static array and a dynamic binary tree. [3]

(d) What is the purpose of a base case in recursion? [2]

Question 7 - Programming [12 marks]

(a) Distinguish a function from a procedure. [2]

(b) Explain encapsulation, inheritance and polymorphism. [3]

(c) Distinguish passing a parameter by value from passing by reference. [2]

(d) Write pseudocode to input 10 integers and output the largest. [5]

Question 8 - Databases [8 marks]

(a) Explain the purpose of a primary key and a foreign key. [2]

(b) State two requirements of First Normal Form (1NF). [2]

(c) A table has composite key (StudentID, CourseID), but StudentName depends only on StudentID. Explain what normalisation change is required to reach 2NF. [2]

(d) Write an SQL query to display StudentName for all students in FormLevel 6, in alphabetical order. [2]

Question 9 - Web application development [8 marks]

(a) State the roles of HTML, CSS and JavaScript. [3]

(b) Explain why server-side validation is necessary even when client-side validation is used. [2]

(c) Describe the steps that occur when a web application retrieves a record from a database. [3]

Question 10 - Enterprising [8 marks]

(a) Distinguish e-business from e-commerce. [2]

(b) State two forms of intellectual property protection and what each protects. [2]

(c) Identify two sections of an ICT business proposal and explain why each is important. [4]

Theory practice: suggested answers and marking points

Award marks for equivalent technically correct answers. The points below are a guide for self-marking and classroom discussion.

Q1 [12]

(a) $2F_{16} = 2\times16 + 15 = 47_{10}$; binary = 0010 1111. [4]
(b) +23 = 00010111; invert = 11101000; add 1 = 11101001. [3]
(c) Mantissa stores significant digits; exponent determines binary-point position / scale. [3]
(d) Overflow: result exceeds representable range; underflow: non-zero result is too small in magnitude to represent. [2]

Q2 [12]

(a) Von Neumann shares memory / path for data and instructions; Harvard separates instruction and data memory / pathways. Award two paired differences. [4]
(b) Address bus carries location; data bus carries data; control bus carries signals such as read/write. [3]
(c) PC: next instruction address; MAR: memory address; MDR: data being transferred; CIR: current instruction. [4]
(d) Overlaps stages of different instructions and can improve throughput. [1]

Q3 [12]

(a) Application, Presentation, Session, Transport, Network, Data Link, Physical. [3]
(b) Resolves a domain name to an IP address. [2]
(c) Public addresses are globally routable; private addresses are for internal networks and are not routed globally. [2]
(d) RIP is distance-vector and uses hop count; OSPF is link-state and calculates shortest paths using cost. [3]
(e) SaaS supplies ready-to-use software; IaaS supplies virtualised infrastructure such as computing, storage and networking. [2]

Q4 [8]

(a) Any four accurately described stages with a suitable output: feasibility report, requirements specification, system design, code / modules, test report, installed system / user training, maintenance updates. [4]
(b) Benefits: early user feedback; clarify requirements; test interface / usability. Limitations: scope creep / repeated changes; prototype may be mistaken for finished product; throwaway code may be poor quality. [4]

Q5 [8]

(a) Confidentiality restricts access; integrity protects accuracy / completeness; availability ensures authorised access when needed. [3]
(b) Phishing / social engineering. Controls: do not follow the link; report / verify via a trusted channel; MFA; user awareness; email filtering. [3]
(c) Tested backups; recovery procedures; alternate equipment / site; contact list; priorities and restoration order. Any two. [2]

Q6 [12]

(a) Midpoints: index 3 = 18; search right half, midpoint index 5 = 31. The target is found. Give the sequence of interval reduction. [4]
(b) First pass: [4, 7, 2, 9]. [3]
(c) Array usually fixed-size and indexed; binary tree is node-linked / dynamic and hierarchical, with at most two children per node. [3]
(d) Base case stops recursion and prevents endless calls. [2]

Q7 [12]

(a) A function returns a value; a procedure performs an action and need not return a value. [2]
(b) Encapsulation bundles data and methods / controls access; inheritance derives from a parent class; polymorphism allows same interface or method name to behave differently. [3]
(c) By value passes a copy; by reference allows changes to affect the caller's variable. [2]
(d) Example:

```
largest ← -∞
FOR i ← 1 TO 10
  INPUT x
  IF x > largest THEN largest ← x ENDIF
NEXT i
OUTPUT largest
```

[5]

Q8 [8]

(a) Primary key uniquely identifies a row; foreign key references a key in another table. [2]
(b) Atomic values in each cell; no repeating groups; unique rows. Any two. [2]
(c) Remove StudentName from the composite-key table into STUDENT keyed by StudentID, so each non-key field depends on the whole key. [2]
(d) SELECT StudentName FROM Student WHERE FormLevel = 6 ORDER BY StudentName; [2]

Q9 [8]

(a) HTML structures content; CSS controls presentation; JavaScript adds behaviour / interactivity. [3]
(b) Browser-side checks can be disabled or bypassed; server must independently validate untrusted requests before processing / storing. [2]
(c) Browser request → server validates/authenticates → server queries database → database returns record → server formats response → browser displays it. [3]

Q10 [8]

(a) E-business covers broader digital business processes; e-commerce is electronic buying and selling. [2]
(b) Copyright protects original expression / software; trademark protects brand identifiers; patent protects qualifying inventions; trade secret protects confidential commercial information. Any two with explanation. [2]
(c) E.g. market analysis identifies customers and competitors; financial plan tests viability; marketing plan explains customer acquisition; operations plan explains delivery. Two sections with purpose. [4]

Self-marking For explanation questions, award full credit only when the learner gives both the correct concept and a relevant explanation. Accept other valid examples.

Practical examination preparation

The current syllabus describes a 3-hour practical paper with three sections. Candidates answer one question from each section: Web Application Development (20 marks), Programming and Algorithm Design (50 marks), and Databases (30 marks). The tasks below are original practice tasks modelled on those skill areas.

Practical workflow Read the full task first. Save files in a clearly named folder. Test each feature. Keep backup copies. Print or export the required evidence and ensure names / candidate details are correct.

Section A - Web application development [20 marks]

A community learning centre needs a two-page website: Home and Course Registration. Build the website using HTML, CSS and JavaScript.

- Home page: centre name, welcome heading, overview paragraph, list of courses and navigation links to both pages. [5]
- Registration page: form with applicant name, date of birth, email, phone number, course selection, study mode and submit button. Use suitable labels and input types. [7]
- Apply consistent CSS styling, readable layout and responsive design. [3]
- Add client-side validation for required fields and a sensible email format. [2]
- Test navigation and form behaviour; provide screenshots / printed evidence and a backup of the files. [3]

Section B - Programming and algorithm design [50 marks]

Develop a Learner Results Manager that stores at least 10 learners. Each record contains LearnerID, Name and marks for three subjects.

- Input and validate learner records. [8]
- Calculate each learner's total and average. [6]
- Assign a grade using a clearly stated grading scale. [6]
- Search for a learner using LearnerID. [6]
- Sort records by average mark in descending order. [8]
- Save records to a file and load them when the application starts. [6]
- Use functions / procedures, arrays or suitable data structures, clear interface design and meaningful error handling. [6]
- Test at least normal, boundary and invalid inputs. [4]

Section C - Databases [30 marks]

Create a database for the learning centre with tables COURSE, LEARNER and ENROLMENT.

- Create the tables with suitable data types, primary keys and foreign keys. [8]
- Enter at least five sample records in each relevant table. [4]
- Write SQL to list all learners, list learners enrolled in a selected course, update a learner's phone number, and delete a specified enrolment. [8]
- Create a query that counts enrolments per course. [4]
- Explain one integrity constraint and one backup / security measure used in the database. [4]
- Demonstrate that the database and queries work; save the database and query evidence. [2]

Practical candidate checklist

- ☐ I can write, test and debug a program using selection, iteration, arrays and subprograms.
- ☐ I can explain and implement a searching or sorting algorithm.
- ☐ I can save and retrieve data from a file.
- ☐ I can build a multi-page website with a working form and validation.
- ☐ I can create relational tables and set primary / foreign keys.
- ☐ I can write SELECT, INSERT, UPDATE and DELETE statements.
- ☐ I can connect application code to a database using safe parameterised queries.
- ☐ I save backups and can demonstrate that the finished work runs correctly.

Final file-submission check

- Open the saved project from the folder you intend to submit, not only from the development environment.
- Check filenames, candidate details, page orientation, headings and printed output.
- Test one normal input, one boundary value and one invalid input for each major feature.
- Confirm that links, forms, SQL statements and program functions work as expected.

School-based project: seven-stage reminder

The syllabus sets out a seven-stage school-based project process. A well-documented project should show how the learner moved from a real problem to an evaluated solution.

Stage	What to demonstrate	Evidence to keep
1. Problem identification	Define the real problem, who is affected and why it matters.	Problem statement, objectives, scope.
2. Investigation	Study current processes and related solutions; collect user requirements.	Questionnaires, interviews, observations, document review.
3. Generate possible solutions	Propose several feasible ways to solve the problem.	Options, sketches, comparison table.
4. Select suitable solution	Compare options against requirements, time, cost and resources.	Selection matrix and justification.
5. Refine solution	Improve the selected design after review and feedback.	Updated design, data structures, interface / logic changes.
6. Present final solution	Demonstrate the completed system / artefact.	Working system, screenshots, user guide, demonstration.
7. Evaluate and recommend	Test objectives, identify strengths / limitations and propose improvements.	Test results, evaluation, recommendations.

Project advice Keep dated evidence as you work. Do not wait until the end to write the project report. Include the problem, requirements, alternatives, design, implementation, testing and evaluation.

Final examination reminders

Theory paper

- Answer every compulsory question and watch the marks allocated to each part.
- For “state” or “identify”, give a concise fact. For “explain”, give the fact plus how / why. For “compare”, address both sides using the same comparison point.
- Show workings for binary conversions, floating-point normalisation, trace tables and algorithm dry runs.
- Use exact terms: PC, MAR, MDR, CIR; primary key, foreign key; authentication, authorisation; client-side, server-side.
- Draw diagrams with titles, clear labels, arrows and a short explanation.

Practical paper

- Attempt one question from each of the three sections, as instructed on the paper.
- Use the required language / software available at the centre. Indicate the programming language used where required.
- Print or export readable evidence, label outputs and include screenshots where requested by the paper.
- Test boundary cases and invalid inputs, not only one successful example.
- Save a working copy and a backup; check that files open correctly before submission.

Syllabus and examination sources

Primary curriculum reference: Ministry of Primary and Secondary Education, Computer Science Syllabus, Forms 5–6, 2024–2030. The revision coverage and assessment summary in this booklet are based on the syllabus scope-and-sequence, competency matrix and assessment sections.

Exam-format cross-check: ZIMSEC Computer Science 6023/1 and 6023/2 specimen paper descriptions. This booklet’s questions and practical tasks are original revision exercises, not copied official examination questions.

Online references checked during preparation:

• Syllabus text: https://studylib.net/doc/28464044/revised-computer-science-f5-6

• Syllabus download details: https://sytbay.co.zw/downloads/zimsec-a-level-computer-science-syllabus-2024-2030-hbc-pdf/

• Paper 1 specimen listing: https://www.scribd.com/document/1057029310/6023q01-Computer-Science

• Paper 2 specimen listing: https://www.scribd.com/document/1054798049/6023q02-Computer-Science

Important Always check any amendments or circulars issued by ZIMSEC / the Ministry. This booklet is a study aid, not a replacement for the complete official syllabus.